

Names: Kwok David, Yu Wing Ching SIDs: 1155174302, 1155175846

IK2406 Craft Your Personal Language Model for Tailored Chatbots

Supervised by Professor Irwin King

Abstract

This project focuses on the development of an advanced RAG system aimed at significantly improving question-answering accuracy. Following a comprehensive evaluation of system advancements, application has been deployed to demonstrate the effect of these advancements.

For system perspective, a comprehensive research methodology is employed, encompassing a thorough literature review, implementation of diverse approaches, and rigorous evaluation of these methods. By addressing the limitations of traditional naive RAG systems, this study explores innovative strategies to elevate the performance of question-answering mechanisms.

Beyond system development, applications were built upon this integrated framework, incorporating functionalities such as a general chatbot, document-based conversational capabilities, and resume screening functionality.

Furthermore, this project serves as a culmination of my academic journey, integrating foundational knowledge in computer science to demonstrate practical applications and problem-solving methodologies acquired over the past four years.

Personal Contributions.....	4
1. Introduction.....	5
1.1. Background	5
1.2. Problem Statement	5
2. Literature Review.....	5
2.1. LLM	5
2.1.1. Text Embedding	6
2.1.2. Text Generation	6
2.1.3. Hallucination	6
2.2. Retrieval-Augmented Generation (RAG)	7
2.3. Database (External Knowledge Base).....	7
2.3.1. Vector Database.....	7
2.4. Information Retrieval	7
2.5. Prompt Engineering.....	7
2.6. Chain-Of-Thought (CoT)	8
Part I, II. System Integration: Implementation and Evaluation	8
3.0. LLM Foundation	8
3.1. RAG Foundation	10
3.2. Evaluation Framework	11
3.2.1. CRAG benchmark [3].....	11
3.2.2. RAGAS metrics [16]	11
3.2.3. LLM as Judge [19]	12
3.3. Evaluation Process	12
3.3.1. Test Case Construction.....	12
3.3.2. Example Test Case in JSON Objects (Simplified).....	12
3.3.3. Performance Measurement	13
3.3.4. Error Metrics.....	13
3.4. Final Experiment and Improvements	13
3.5. Parameter Optimization.....	14
3.5.1. Embedding Model Selection	14
3.5.2. Optimization Process.....	15

3.6. Advance Retrieval	16
3.7. Integrated Knowledge Graph (KG) Database	17
3.8. Chain Of Thought (CoT).....	21
3.9. Retrieval Interleaved Generation (RIG).....	21
3.10. Evaluation of Integrated RAG System.....	23
Part III. Application (Term2).....	23
4.1. Use Cases and Functionalities.....	23
4.1.1. Chatbot Page.....	23
4.1.2. Resume Screening Page.....	24
4.2. Frontend development.....	24
4.2.1. Frontend development Timeline.....	24
4.3. User flow	25
4.4. Backend development	33
4.4.1. Document Management.....	33
4.4.2. Collection Management for Conversations	33
4.4.3. Conversation Management	33
4.4.4. Resume Screening Logic Design.....	34
4.4.5. Evaluation of Resume Screening System	36
5. Future Directions	37
6. Conclusion	38
7. Capstone.....	38
Github repository	41
References.....	41

Personal Contributions

Contributor	Term	Tasks
David	Term1	Integrate LLM with RIG
		Evaluate the performance with benchmarks
		Section 3.2, 3.3, 3.5, 3.7
Winnie	Term1	Integrate LLM with IR
		Section 1, 2, 3.0, 3.1, 3.4
David	Term2	Develop backend of application on top of the integrated LLM
		Evaluate the performance with benchmarks
		Section 3.8, 4.4
Winnie	Term2	Design UI for the application
		Develop frontend of application on top of the integrated LLM
		Section 4.1, 4.2, 4.3, 5, 6

Throughout the course of this Final Year Project (FYP), numerous obstacles were encountered, presenting both challenges and opportunities for growth. In the first term, I collaborated with David to integrate the RAG system using various methods. This involved extensive research, implementing Information Retrieval techniques, and conducting part of the evaluation process. These efforts laid the foundation for understanding the potential of RAG systems and their practical applications.

Moving into the second term, I initiated the development of the application prototype, focusing on an interface with multiple chatbot contexts. These included course-specific, domain-specific, and CUHK-related chatbots, allowing users to pose relevant questions. The system utilized both in-built documents specific to private domains and documents uploaded by users, ensuring updated and accurate information. A Figma file was created for prototype design, and the UI was built using Minimal, a package developed on top of the MUI library. However, the project's direction shifted to prioritize resume screening functionality, necessitating a near-complete redesign of the UI.

The new interface, detailed in Part III of this report, integrates both chatbot querying and resume screening features. Screenshots have been attached to showcase the design, and the code has been committed to a private GitHub repository. Debugging was performed once backend endpoints became available, ensuring functionality, although there remains room for further improvements.

Since basic users of Figma are not allowed to share the document to public, so I attached the code below so that the initial design could be reachable with html renderer without user login and permission. `<iframe style="border: 1px solid rgba(0, 0, 0, 0.1);" width="800" height="450" src="https://embed.figma.com/proto/ARaouG8aYBXsCbLNppmXAf/FYP?node-id=4274-33916&p=f&scaling=min-zoom&content-scaling=fixed&page-id=866%3A55474&embed-host=share" allowfullscreen></iframe>`

1. Introduction

1.1. Background

Pre-trained large language models (LLMs) are widely utilized across various domains, demonstrating capabilities in natural language understanding and generation. However, hallucination, as one of the most critical limitations, is still inevitable that LLMs often fail to provide accurate responses regarding up-to-date, private or domain-specific knowledge. [1] To tackle the phenomena of hallucination, Retrieval Augmented Generation (RAG) has been introduced. RAG enhances LLM through Retrieving relevant contexts from external knowledge base through retrieving algorithms, Augmenting retrieved chunks to original prompt, Generating output through LLM with updated prompt. [2]

1.2. Problem Statement

Straightforward RAG solution, often referred to as naive RAG, demonstrates low accuracy performance. [3]

1.3. Motivation

To overcome limitations of naive RAG, the integration of advanced retrieval and augmentation techniques is essential. This project is driven by the ambition to refine and enhance RAG systems, aiming for substantial improvements in question-answering accuracy through rigorous methodologies and quantitative evaluation. Ultimately, the goal is to develop a high-performance RAG system capable of delivering more precise and context-aware answers.

2. Literature Review

2.1. LLM

Large Language Models (LLMs) represent a significant evolution in natural language processing (NLP). According to the paper "A Survey of Large Language Models"[4], NLP has evolved through several stages:

- **Statistical Language Models (SLMs):** Early models relied on statistical techniques to estimate the probability of word sequences, forming the basis of probabilistic language understanding.
- **Neural Language Models (NLMs):** With advancements in deep learning, architectures like neural networks (NN), Recurrent Neural Networks (RNNs), and Multi-Layer Perceptron (MLP) introduced more sophisticated representations of linguistic structures.
- **Pre-trained Language Models (PLMs):** These models leveraged massive datasets for pre-

training and were then fine-tuned for specialized tasks, significantly enhancing NLP performance across various domains.

The emergence of LLMs represents the scaling of PLMs to unprecedented sizes. Through extensive training on vast data sets, LLMs exhibit emergent abilities that were not observed in smaller models. Notable among these abilities are:

- **In-Context Learning:** The ability to dynamically adapt to new contexts without explicit retraining.
- **Instruction Following:** Enhanced ability to process complex prompts, improving responsiveness across diverse tasks.
- **Step-by-Step Reasoning:** The capacity to generate coherent, multi-step explanations, advancing applications in structured problem-solving and logical reasoning.

2.1.1. Text Embedding

Text embedding is a fundamental technique in NLP that transforms human language into a structured, computer-understandable format. According to a paper reviewing top performing embedding method [5], high-quality embeddings are critical for various NLP tasks, including information retrieval and semantic textual similarity.

The efficacy of LLM applications largely depends on high-quality text embeddings that these embeddings play a pivotal role in tasks like vector search, where relevant documents are retrieved based on their similarity to a query. This process is essential for LLM-based question-answering (QA) systems, ensuring that the most contextually relevant information is retrieved to generate accurate responses.

2.1.2. Text Generation

LLM possesses the ability to generate text which was a challenging task in NLP. With advancement on LLM, text generated become more coherent, context-aware, and sophisticated which facilitates machine translation, text summarization and question answering relevancy. [6]

2.1.3. Hallucination

Since Pre-trained Language Models (PLMs) have scaled into Large Language Models (LLMs), hallucination such as relevant but factually incorrect, logically irrelevant output has been observed. Several factors contribute to this phenomenon, including the ingestion of poor-quality, biased, outdated, or misinformed data during training. These limitations affect an LLM's ability to provide precise and reliable information, particularly in critical applications such as

knowledge retrieval and decision-making systems . [1]

2.2. Retrieval-Augmented Generation (RAG)

To mitigate hallucinations, RAG has been introduced to improve accuracy and credibility of output generation. During retrieval stage, relevant documents of user input will be extracted from external knowledge base. Then, chunks will be concatenated to original user input with prompts. Finally, output will be generated with input of enhanced query through LLM. Retrieval guarantees context relevancy, so to reduce hallucinations. [2]

2.3. Database (External Knowledge Base)

2.3.1. Vector Database

Vectors are commonly perceived as embedded representations of data that encapsulate high-dimensional features. Only vector databases can effectively store, index, and retrieve unstructured data. [6]

2.4. Information Retrieval

Information retrieval plays a vital role in efficiently retrieving relevant information based on user queries, ensuring the context provided aligns with their information needs. However, several challenges, including query ambiguity, can impede the efficiency of the retrieval process.

To address these challenges, an IR system typically consists of multiple components designed to enhance retrieval accuracy:

- **Query Rewriter:** Refines and optimizes user queries to improve precision and recall, making searches more effective.
- **Retriever:** Extracts relevant documents based on user queries. A widely used model in information retrieval is the classical **bag-of-words approach**, such as **BM25 (Best Matching 25)**, which ranks documents based on their relevance to the input.
- **Reranker:** Reorders retrieved documents by evaluating additional relevance criteria, filtering out redundant or overlapping contexts to ensure the most useful information is prioritized. [7]

2.5. Prompt Engineering

Prompt engineering aims to optimize the capabilities of pre-trained large language models (LLMs). It involves strategically crafting task-specific instructions, known as prompts, to guide

model output without modifying underlying parameters.

By leveraging well-structured prompts, prompt engineering provides a flexible mechanism for improving model responses across diverse tasks and domains. This adaptability is different from traditional paradigms, where model retraining or extensive fine-tuning is often required for task-specific performance. [8]

2.6. Chain-Of-Thought (CoT)

Chain-of-Thought is a reasoning technique designed to enhance the ability of language models to generate a coherent series of intermediate reasoning steps leading to the final answer for complex problems.

By breaking down multi-step problems into intermediate steps, enabling additional computation to be allocated to tasks that require deeper reasoning. This transparency can aid in debugging and understanding the model's decision-making process.

CoT can be effectively implemented using few-shot prompting, where examples of chain-of-thought reasoning sequences are incorporated into training data. Notably, this technique can be integrated into existing models without significant architectural modifications, making it a scalable and efficient method for enhancing logical reasoning capabilities. [9]

Part I, II. System Integration: Implementation and Evaluation

In the first semester, the project focused on system integration, beginning with a thorough literature review to establish foundational understanding. This research phase involved analyzing newly introduced frameworks, clarifying concepts, identifying key problems, and narrowing the project scope to define actionable directions for integration.

Following this, specific techniques were selected and implemented to enhance system performance. The necessity and impact of these techniques were systematically evaluated, with benchmarking used to validate improvements. The focus was on optimizing Large Language Models (LLMs) through efficient solutions, prioritizing minimal resource usage while maintaining high accuracy and functionality. By refining methodologies and selecting the most effective approaches, the project aimed to develop an elegant, streamlined system capable of achieving robust performance.

3.0. LLM Foundation

Analysis of the Chatbot Arena leaderboard [10], a community-driven evaluation platform for LLMs and AI chatbots, revealed that OpenAI's GPT series consistently maintains top rankings.

Other notable performers include Anthropic's Claude and open-source alternatives such as Google's Gemini and NVIDIA's LLaMa.

Our implementation strategy prioritized building upon established high-performance models rather than developing a foundation model from scratch. [11] While initial experiments with local deployment of open-source models like LLaMa were conducted, hardware constraints, limited GPU resources, led to the adoption of Microsoft Azure OpenAI API services, selected for its reliability and regional support in Hong Kong.

Azure OpenAI Implementation

The implementation required three key components:

1. API Key authentication
2. Endpoint configuration
3. Model version selection (GPT-4o)

The system architecture enables:

- Query processing (user inputs)
- Response generation
- Role definition capabilities
- Output length control

Sample Code for Microsoft Azure OpenAI API service [12]

```
from openai import AzureOpenAI

client = AzureOpenAI(
    azure_endpoint = os.getenv("AZURE_OPENAI_ENDPOINT"),
    api_version="2024-09-01-preview"
)

response = client.chat.completions.create(
    model="o1-preview-new", # replace with the model deployment name of your o1-preview, or o1-mini model
    messages=[
        {"role": "user", "content": "What steps should I think about when writing my first Python API?"},
    ],
    max_completion_tokens = 5000
)

print(response.model_dump_json(indent=2))
```

3.1. RAG Foundation

To address hallucination issues in LLM outputs, we implemented RAG methodology. While both LangChain [13] and LlamaIndex frameworks were evaluated, LlamaIndex was selected for its comprehensive documentation and gentler learning curve.

The **RAG system architecture** consists of three key components:

- **Vector database** construction with embedded documents
- **Context retrieval** mechanism based on user queries
- **Enhanced response generation** incorporating retrieved context

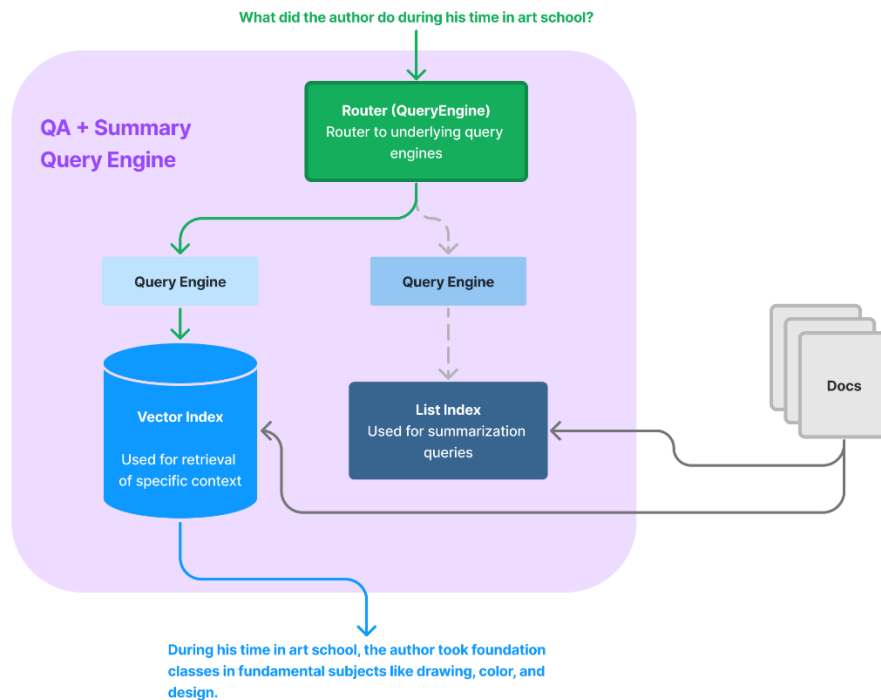


Fig1. Workflow of Dynamically Retrieve Chunks Depending on Task [14]

By establishing this approach, RAG effectively acts as a “reference system” for the LLM, significantly improving response accuracy and contextual relevance. [2] The next phase involves developing an **evaluation framework** to quantify the performance improvements achieved through RAG implementation.

3.2. Evaluation Framework

The evaluation framework was designed to assess system performance, comprising three key components:

3.2.1. CRAG benchmark [3]

Utilized dataset of Task #3 [15], a “.jsonl” file containing 2706 entries, each with

- **query**: string formatted user question
- **answer**: string formatted gold-standard expected response
- **search_results**: array of JSON formatted HTML pages (around 50 pages per question)

3.2.2. RAGAS metrics [16]

Evaluate the qualities of retrieved contexts using **Context Precision** (Fig.2) and **Context Recall** (Fig.3):

- **Context Precision** [17]: Measures whether expected-answer-relevant chunks are prioritized.

This metric is computed using the `question`, `ground_truth` and the `contexts`, with values ranging between 0 and 1, where higher scores indicate better precision.

$$\text{Context Precision@K} = \frac{\sum_{k=1}^K (\text{Precision@k} \times v_k)}{\text{Total number of relevant items in the top } K \text{ results}}$$
$$\text{Precision@k} = \frac{\text{true positives@k}}{(\text{true positives@k} + \text{false positives@k})}$$

Where K is the total number of chunks in `contexts` and $v_k \in \{0, 1\}$ is the relevance indicator at rank k .

Fig.2 Formula to calculate Context Precision

- **Context Recall** [18]: Assesses alignment of retrieved contexts with the expected answer.

The formula for calculating context recall is as follows:

$$\text{context recall} = \frac{|\text{GT claims that can be attributed to context}|}{|\text{Number of claims in GT}|}$$

Fig.3 Formula to calculate Context Recall

3.2.3. LLM as Judge [19]

Compares the actual RAG output to the expected answer, marking responses as **true/false**. This technique employs **prompt engineering**, including a system prompt and few-shot prompting, adapted from **Meta Comprehensive RAG Benchmark (KDD Cup 2024)** [20].

3.3. Evaluation Process

The LLM as Judge program was validated through the following steps:

3.3.1. Test Case Construction

- Each test case consists of a **list of 10 questions** (one CRAG benchmark question + **9 paraphrased variants** generated by GPT-4o).
- Each question has a **ground truth answer** and **10 predicted answers** output from the RAG system.
- A manually counted **supposed_num_of_true** value (range **0–10**) represents how many answers correctly match the ground truth.
-

3.3.2. Example Test Case in JSON Objects (Simplified)

Test Case 1:

```
{“id”: 1,  
“questions”: [“What is the full name of CUHK?”, “full name of CUHK”, “Can you provide the full name of  
CUHK?”, “complete name of CUHK”, “What is the whole name of CUHK?”, “What does CUHK stands  
for?”, “CUHK stands for what?”, “What does CUHK mean?”, “What is the meaning of CUHK?”, “What is  
CUHK’s full name?”],  
“ground_truth”: “The Chinese University of Hong Kong”,  
“answers”: [“The Chinese University of Hong Kong”, “CUHK”, “The China University of Hong Kong”,  
“CUHK”, “The Chinese Universe of Hong Kong”, “CUHK”, “CUHK”, “The Cheerful University of Hong  
Kong”, “CUHK”, “CUHK”],  
“supposed_num_of_true”: 1}
```

Test Case 2:

```
{“id”: 2,  
“questions”: [“Is David a student of The Chinese University of Hong Kong?”, “Does David study at CUHK?”,  
“Does CUHK have a student named David?”, “Does David study at The Chinese University of Hong Kong?”,  
“Does The Chinese University of Hong Kong have a student named David?”, “Is David enrolled in CUHK?”,  
“Is David enrolled in The Chinese University of Hong Kong?”, “Is David one of the student who studies at
```

```
CUHK?", "Is David one of the student who studies at The Chinese University of Hong Kong?", "Is David one of CUHK's student?"]],
"ground_truth": "Yes",
"answers": ["Yes", "No", "Yes", "Yes", "No", "No", "No", "No", "No", "No"],
"supposed_num_of_true": 3}
```

3.3.3. Performance Measurement

For each test case, the **LLM as Judge** determines correctness of each answer by comparing with the ground truth and assigning a **supposed_num_of_true_llm** value by counting the number of matched answers. Example output:

```
[
  {"id": 1, "supposed_num_of_true": 1, "supposed_num_of_true_llm": 1},
  {"id": 2, "supposed_num_of_true": 3, "supposed_num_of_true_llm": 4}
]
```

3.3.4. Error Metrics

- Mean Absolute Error (MAE) = average of $|\text{supposed_num_of_true} - \text{supposed_num_of_true_llm}|$
- Root Mean Square Error (RMSE): $\sqrt{\text{mean}((\text{supposed_num_of_true} - \text{supposed_num_of_true_llm})^2)}$

Example calculations for test case above:

- $\text{MAE} = (|1 - 1| + |3 - 4|) / 2 = \mathbf{0.5}$
- $\text{RMSE} = \sqrt{((1 - 1)^2 + (3 - 4)^2) \div 2} = \mathbf{0.7071}$

3.4. Final Experiment and Improvements

Initial evaluation using 47 test cases resulted in:

- MAE: 1.43
- RMSE: 3.47

However, test cases with ground truth content as **"invalid question"** caused inconsistencies, as RAG outputs often provided context-based explanations rather than a direct match. To address this:

- System prompt modifications were implemented.
- Few-shot prompting examples were extended for improved performance.

A follow-up experiment with 50 "invalid question" cases showed enhanced results:

- MAE: 0.22
- RMSE: 0.97

With near-zero values in MAE and RMSE, the **LLM as Judge** program demonstrated reliability in evaluating RAG-generated responses.

3.5. Parameter Optimization

Optimizing the parameters of the Retrieval-Augmented Generation (RAG) system involved tuning key components to improve retrieval efficiency and response accuracy. The following parameters were systematically adjusted:

- Embedding Model: Determines how text is transformed into vector representations.
- Chunk Size & Chunk Overlap: Controls document segmentation before indexing in the vector database.
- Similarity Top-K: Defines the number of retrieved chunks per query.
- Distance Metric: Measures similarity between embedding vectors.

3.5.1. Embedding Model Selection

To optimize embedding quality, the Massive Text Embedding Benchmark (MTEB) leaderboard [21, 22] was referenced (Fig.4). While bge-en-icl ranked highest, hardware constraints (48GB GPU memory) prevented its use due to excessive model size (~7B parameters, causing "CUDA out of memory" errors). Consequently, the second-best model, stella_en_1.5B_v5, was selected, providing a balance between performance and computational feasibility (~1.5B parameters).

Overall Bibtex Mining Classification Clustering Pair Classification Reranking **Retrieval** STS Summarization Retrieval w/Instructions

Retrieval is the task of finding relevant documents for a query.

English Chinese French Law LongEmbed Polish Russian RAR-b BRIGHT

Retrieval English leaderboard

- Metric: Normalized Discounted Cumulative Gain @ 10 (nDCG@10)
- Languages: English

Rank	Model	Model Size (Million Parameters)	Memory Usage (GB, fp32)	Average	ArguAna	ClimateFEVER	CQADupstackRetrieval	D
1	bge-en-ic1	7111	26.49	62.16	83.08	45.43	47.31	5
2	stella_en_1.5B_v5	1543	5.75	61.01	65.27	46.11	47.75	5
3	NV-Retriever-v1	7111	26.49	60.9	68.28	43.47	49.36	5
4	gte-Qwen2-7B-instruct	7613	28.36	60.25	64.27	45.88	46.43	5
5	Ling-Embed-Mistral	7111	26.49	60.19	69.65	39.11	47.27	5
6	SFR-Embedding-2_R	7111	26.49	60.18	62.34	34.43	46.11	5
7	NV-Embed-v1	7851	29.25	59.36	68.2	34.72	50.51	4
8	bge-multilingual-gemma2	9242	34.43	59.24	77.37	39.37	47.94	5
9	SFR-Embedding-Mistral	7111	26.49	59	67.17	36.41	46.49	4
10	stella_en_400M_v5	435	1.62	58.97	64.24	43.53	44.36	4

Fig.4 Massive Text Embedding Benchmark (MTEB) leaderboard

3.5.2. Optimization Process

Instead of testing all parameter combinations across the entire **CRAG benchmark dataset (2,706 test cases)**, an iterative approach was used to **control API usage and reduce cost**:

i. Baseline Parameter Set:

Distance Metric: "l2" (Squared L2 in Chroma[D9])

Chunk Size: 1024

Similarity Top-K: 5

Chunk Overlap: $\text{int}(0.2 * \text{chunk size})$

ii. Sequential Parameter Optimization:

Randomized test case selection (100 cases initially).

Iterative evaluation across increasing batches of test cases (100 → 200 → 300 cases).

Optimization order: distance metric → chunk size → similarity top-k → chunk overlap.

Performance assessment via F1 scores (RAGAS metrics) and LLM as Judge scoring.

iii. Final Optimized Parameters:

Distance Metric: "cosine" (Cosine similarity in Chroma)

Chunk Size: 3072

Similarity Top-K: 6

Chunk Overlap: Enabled

Evaluation demonstrated notable improvement:

- Initial LLM as Judge Score (200 test cases): 0.285
- Final Optimized Score: 0.37

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Fig.5 F1 calculation formula

3.6. Advance Retrieval

The project explores advanced retrieval methodologies through RAG Fusion [23] implementations.

In the RAG Fusion approach, the system refines user input by generating 3 semantically related questions, enabling broader retrieval coverage. This method ensures that relevant information is extracted from an external knowledge base with greater precision.

Process Overview

- Query Expansion: The system dynamically generates additional related queries based on user input.
- Multi-Query Retrieval: Each generated query fetches relevant contexts independently.
- Reranking Mechanism: Retrieved results undergo relevancy-based evaluation before integration into response generation.

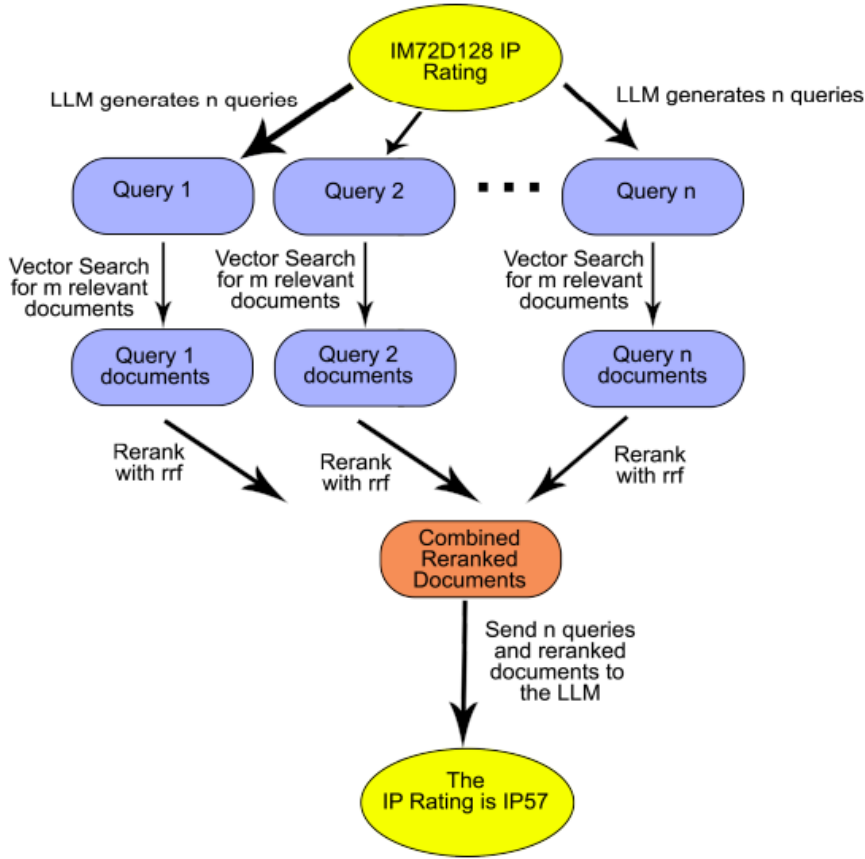


Fig. 6 Diagram illustrating the high-level workflow of RAG-Fusion, starting with the original query” IM72D128 IP Rating” [23]

Once the system retrieves relevant contexts for the generated queries, a reranking process is initiated (Fig.7). This step refines the ranking order using additional scoring metrics, ensuring improved accuracy. The reranked contexts are then preserved for subsequent procedures, maintaining consistency for future analysis and downstream tasks.

$$\text{rrfscore} = \frac{1}{\text{rank} + k}$$

Fig.7 rrfscore calculation formula

3.7. Integrated Knowledge Graph (KG) Database

The project incorporates a knowledge graph database using Neo4j and the LangChain Cypher module [24, 25] (Fig.8,9) to explore an alternative retrieval mechanism for RAG systems.

Compared to vector databases, knowledge graph-based retrieval presents three fundamental differences:

1. Data Structuring Overhead

Since CRAG benchmark data consists of unstructured web pages, an additional preprocessing step is required—converting split chunks into structured graph documents using LLM.

This process is computationally intensive, with data transformation taking up to 3063.04s per test case (~22.04s per chunk, size = 3072).

2. Querying with Cypher Statements

Unlike vector-based retrieval, queries must first be translated into Cypher statements (similar to SQL) by LLM before accessing the database.

3. Direct Response Generation within Cypher Module

Once retrieval results are obtained, the LLM directly generates an answer within the Cypher module, removing the need for additional post-processing.

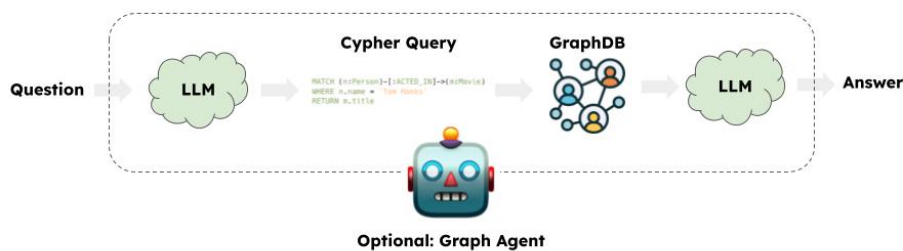


Fig.8 LangChain GraphRAG workflow

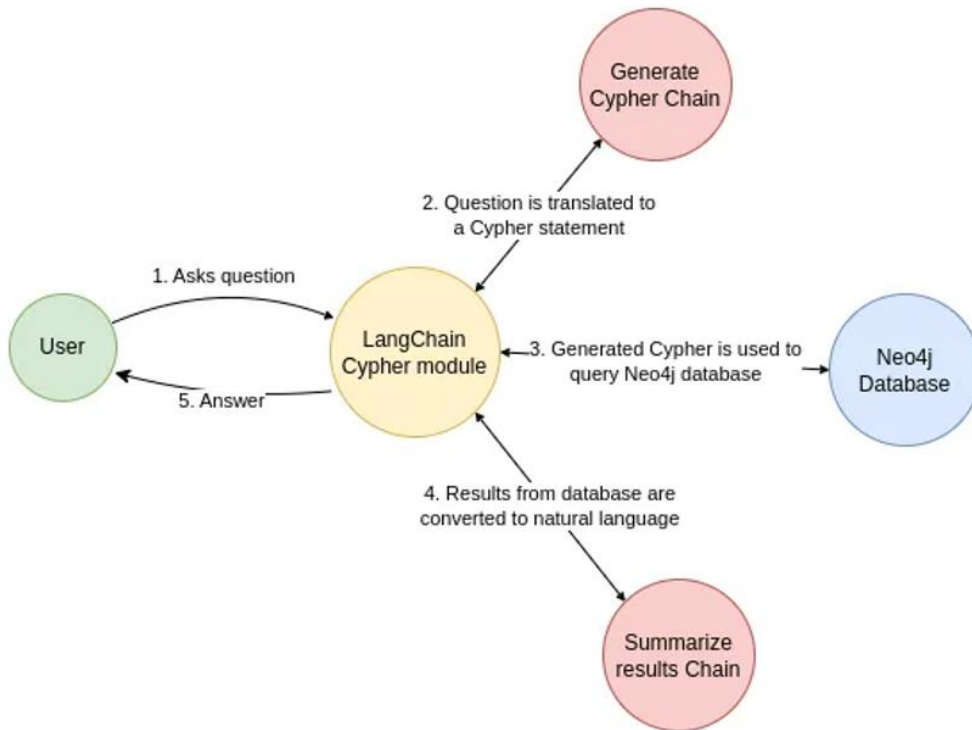


Fig.9 Flow chart illustrates the process by which a user's question is handled using the LangChain Cypher module to generate an answer.

Challenges & Failed Solutions

One major issue observed is property name mismatches in Cypher queries. Example in Fig.10 shows a Cypher statement referencing a "title" property that does not exist in the database, resulting in a failed retrieval ("I don't know the answer").

Attempted Solutions

1. Validating & Correcting Relationship Directions

The "validate_cypher=True" argument was tested to ensure relationship correctness.

However, this only validated relationship directions, not property names (e.g., "reignStart" in Fig.11 remained unverified).

2. Reducing Chunk Size for a More Granular Graph

By decreasing chunk size, more properties should theoretically be captured.

However, a larger graph resulted in model failures—LLM struggled to generate Cypher queries when database complexity increased (Fig.12).

Additionally, data insertion time increased significantly, reducing system efficiency.

```
> Entering new GraphCypherQAChain chain...
Generated Cypher:
cypher
MATCH (m:Movie)
WHERE m.title IN ["The Godfather", "Schindler's List"]
RETURN m.title, m.rating
ORDER BY m.rating DESC
LIMIT 1

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: title)} {position: line: 3, column: 9, offset: 31} for query: 'cypher\nMATCH (m:Movie)\nWHERE m.title IN ["The Godfather", "Schindler's List"]\nRETURN m.title, m.rating\nORDER BY m.rating DESC\nLIMIT 1\n'
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: title)} {position: line: 4, column: 10, offset: 87} for query: 'cypher\nMATCH (m:Movie)\nWHERE m.title IN ["The Godfather", "Schindler's List"]\nRETURN m.title, m.rating\nORDER BY m.rating DESC\nLIMIT 1\n'
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: rating)} {position: line: 4, column: 19, offset: 96} for query: 'cypher\nMATCH (m:Movie)\nWHERE m.title IN ["The Godfather", "Schindler's List"]\nRETURN m.title, m.rating\nORDER BY m.rating DESC\nLIMIT 1\n'
Full Context:
[]

> Finished chain.
{'query': 'which movie is considered the best political film, "the godfather" or "schindler's list"?', 'result': "I don't know the answer."}
```

Fig.10

```
> Entering new GraphCypherQAChain chain...
Generated Cypher:
cypher
MATCH (p:Person)-[:CURRENT HOLDER]->(t:Title)-[:RECOGNIZED BY]->(o:Organization {id: "WBA"})
MATCH (p)-[:HOLDS_LONGEST_REIGN]->(t)
WHERE t.id CONTAINS "world" AND t.id CONTAINS "title" AND t.id CONTAINS "WBA"
AND duration.inDays(p.reignStart, date()).days > 2000
RETURN p.id AS PersonID, t.id AS TitleID, duration.inDays(p.reignStart, date()).days AS ReignDays

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: reignStart)}
```

Fig.11

```
> Entering new GraphCypherQAChain chain...
Generated Cypher:

Full Context:
[]

> Finished chain.
{'query': 'which current wba world title holders have held their reign for over 2000 days?', 'result': "I don't know the answer."}
Generated answers for 1 test cases.
```

Fig.12

Conclusion

Based on these findings, integrating unstructured data into a knowledge graph database presents inherent feasibility issues. A structured knowledge graph, pre-built and stored as a validated data source (Fig.13), could significantly improve RAG system performance, aligning with insights from the CRAG research paper [3].

Task 2: KG and Web Retrieval Augmentation. In Task 2, we in addition provide *mock APIs* to access information from underlying *mock KGs*. The mock KGs store structured data relevant to the questions; answers to the questions may or may not exist in the mock KGs. The mock APIs take input parameters, oftentimes parsed from the question, and provide structured data from the mocked KGs to support answer generation. This task tests how well a RAG system 1) queries structured data sources and 2) synthesizes information from different sources.

Fig.13

3.8. Chain Of Thought (CoT)

The Chain-of-Thought (CoT) methodology plays a crucial role in query reconstruction, enhancing transparency in system reasoning and improving response accuracy. This implementation particularly explores self-ask approaches [26], where LLM-generated intermediate reasoning steps refine contextual understanding.

Evaluation & Performance Metrics

System performance was assessed through step-by-step Zero-shot CoT, Few-shot CoT, and Self-Ask Context methods, with improvements measured using context F1 scores (Ragas) and LLM judge evaluations.

Strategy	Context F1 (RAGAS)	LLM as Judge
Zero-Shot	0.2511	0.27
Few-Shot (Standard)	0.2502	0.32
Few-Shot (Self-Ask)	0.2411	0.34

Further expansion of the dataset to Set 22 (300 cases) confirmed continued improvements in response accuracy, particularly under CoT and Self-Ask Context methodologies.

Dataset Size Impact & Additional Investigations

Evaluations of Self-Ask Accurate Context and Self-Ask Post Context strategies across varying dataset sizes revealed fluctuations in performance, demonstrating dataset-dependent model behavior. Future refinements will focus on optimizing CoT-based reasoning paths to ensure robust scalability and reliability across different test cases.

3.9. Retrieval Interleaved Generation (RIG)

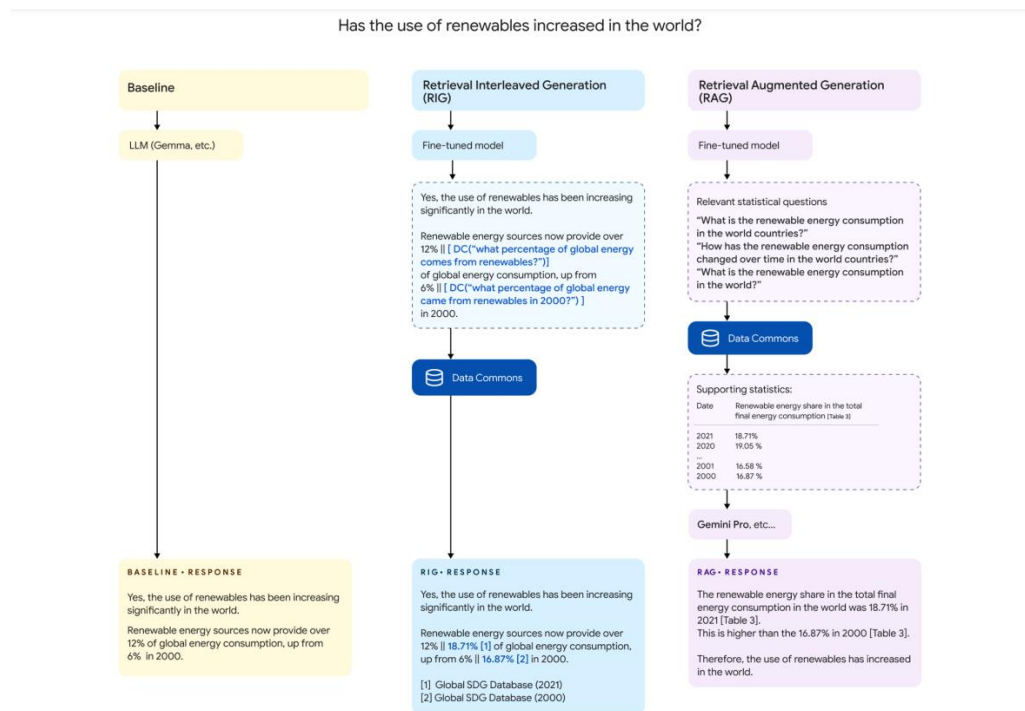


Fig.14[27]

Being inspired by Google with Fig.14, our approach of RIG alone is as follows:

1. The LLM generates an initial answer based solely on user input.
2. The LLM formulates additional questions, each targeting specific details within the generated answer.
3. These generated questions are processed by the RAG system, retrieving relevant information from the knowledge base.
4. If discrepancies arise, the LLM generates corrections to refine inaccurate details.
5. Using both newly verified details and the initial response, the LLM synthesizes a final, improved answer.

RIG was implemented as an extension of RAG, utilizing RAG-generated outputs as inputs to RIG, effectively acting as an error correction layer.

Performance Evaluation

200 test cases

LLM as Judge score (RIG): 0.445 (44.5%)

Improvement over optimized RAG system: +7.5% (previous: 0.37 or 37%)

While RIG demonstrated a measurable improvement, further evaluation was required to determine its reliability.

3.10. Evaluation of Integrated RAG System

The RAG system developed in the last semester, which is integrated with Pre-retrieval, Chain-of-Thought (CoT), and Retrieval Interleaved Generation (RIG), was not evaluated with CRAG benchmark and LLM as judge. In this semester, performances with all combinations of the three techniques implemented were evaluated. The average scores of LLM as judge (True as 1, False as 0) are as follows (corrected to 4 significant digits):

Pre-retrieval	CoT	RIG	Average Score
			0.47
			0.54
			0.5267
			0.4767
			0.5567
			0.5067
			0.5233
			0.5433

A table cell shaded in light grey indicates that the technique has not been implemented. Conversely, a table cell shaded in green signifies that the technique has been implemented.

Therefore, Pre-retrieval + CoT is the best combination. Implementing RIG may negatively impact the response accuracy, and the observed reason is that implementing RIG is essentially performing RAG again for double checking answer details, as RIG includes steps of “retrieve additional contexts for verification” and “synthesize a new, more accurate answer based on verified information”, but since RAG itself has an error rate, there is a probability to synthesize a less accurate answer instead. Thus, RIG is not reliable for improving the system.

Part III. Application (Term2)

This section outlines the use cases and problem-solving features of the application built on the previously introduced integrated system.

4.1. Use Cases and Functionalities

This application offers two primary pages, each addressing distinct use cases:

4.1.1. Chatbot Page

This interface enables users to engage with the chatbot, with or without document uploads. Powered by the integrated model developed in Semester 1, it delivers enhanced accuracy by

improving relevance and ensuring a deeper understanding of context during conversations.

4.1.2. Resume Screening Page

This interface enables users to screen resumes by uploading both job descriptions and resumes. Utilizing the integrated system, it extracts key features that align with the requirements of the hiring position. The uploaded resumes are then analyzed to determine their compatibility with these features. Ultimately, the system identifies top-tier applicants based on their resumes, providing detailed explanations of the reasoning behind their qualification or disqualification.

4.2. Frontend development

The frontend of this application is developed using the **React** framework. The development is carried out in **TypeScript**, a strongly typed superset of JavaScript that enhances code reliability and maintainability. The application uses **Vite** as its build tool and development server, providing a fast and efficient development experience. For components and styling, the **MUI (Material-UI)** library is implemented, offering a range of pre-built, customizable UI components to streamline the design process. Additionally, the interface is designed and implemented responsively, adapting seamlessly to different frame sizes. This ensures enhanced user experience across various devices or window sizes, making the application versatile and user-friendly.

4.2.1. Frontend development Timeline

Initially, the front-end was designed to serve as an interface for user interaction with chatbot functionalities, enabling querying within specific contexts like course-related, domain-specific, or CUHK-related topics. The aim was to allow users to ask questions while utilizing both integrated system documents and user-uploaded files. This phase focused on creating a user-friendly and functional interface to streamline interactions and ensure a seamless experience.

The design process began with a Figma prototype to visualize the user interface and its flow. This prototype provided clarity in design and helped plan the application layout effectively. However, as the backend team developed its demo UI to demonstrate the functionality of various endpoints, significant changes were required. This necessitated undoing most of the frontend progress and adapting to the updated requirements based on the backend demo. While challenging, this shift highlighted the importance of flexibility in development to accommodate evolving needs.

To improve maintainability, the frontend code was restructured using **Vite** and **React**, incorporating strict type restrictions with **TypeScript**. This approach enhanced code reliability

and simplified future updates. Further adjustments involved deciding whether to introduce new routes or manage rendering logic within the same page using tabs, ensuring modularity and efficient management of components.

Updates from the backend, particularly for the resume screening functionality, prompted additional refinement to the frontend. Efforts were directed towards making the interface more user-friendly and visually intuitive while maintaining a clean and consistent flow. The design evolved to balance functionality with aesthetics, retaining the structural integrity of the application without compromising usability.

Beyond frontend development, the backend code was amended to enhance case-checking for edge scenarios with greater precision. This collaborative process between frontend and backend development underscored the importance of alignment and communication within a team. Lastly, the front-end underwent extensive debugging to address functionality issues and integrate seamlessly with the backend endpoints. Although debugging was completed, optimization remains an area for future improvement.

This journey has been an enlightening experience, showcasing the dynamic nature of frontend development and the value of adaptability, collaboration, and attention to detail. It has also reinforced the significance of maintaining a user-centric approach to design and functionality throughout the development process.

4.3. User flow

When users launch the application, the **landing page** (Fig 15) features two buttons, each serving as a gateway to the respective main pages. These buttons provide intuitive navigation, enabling users to seamlessly access either the **Chatbot Interface** (Fig 16-22) or the **Resume Screening Interface** (Fig 23-27), depending on their intended use. This design ensures a straightforward and user-friendly experience right from the start.

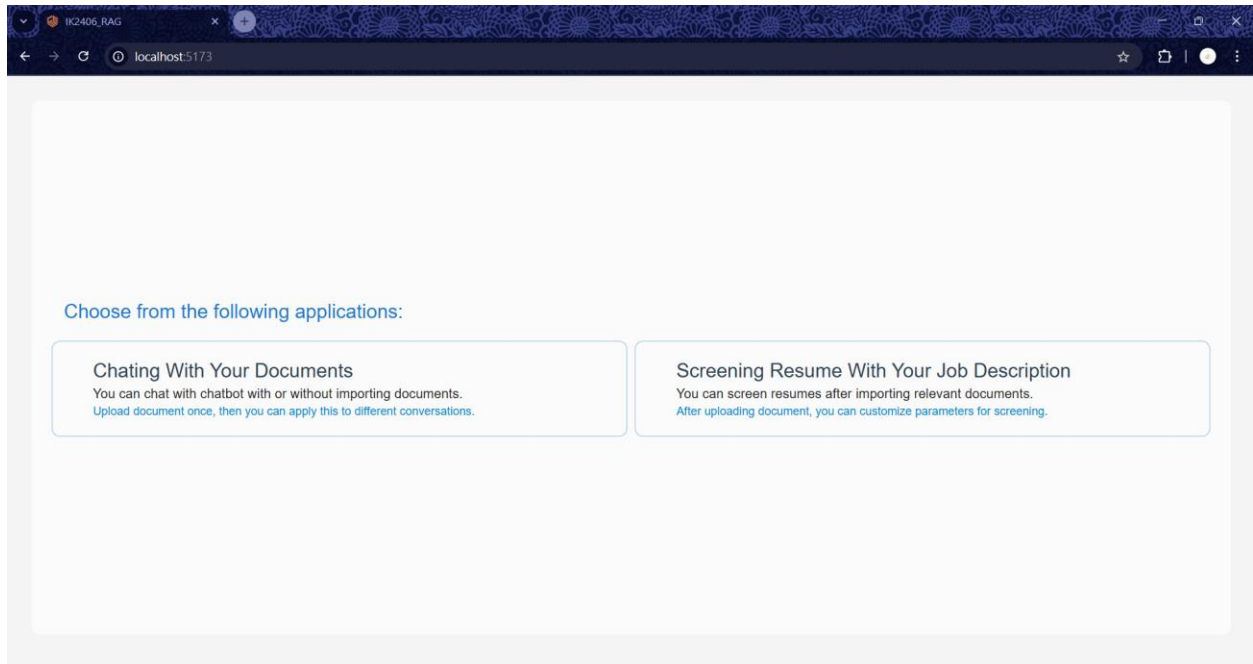


Fig 15

Users can upload documents and access the general database, which stores all uploaded files. The interface (Fig 16) provides functionalities for viewing and managing these documents, including options to delete files directly from the database when needed. This ensures efficient organization and control over stored content.

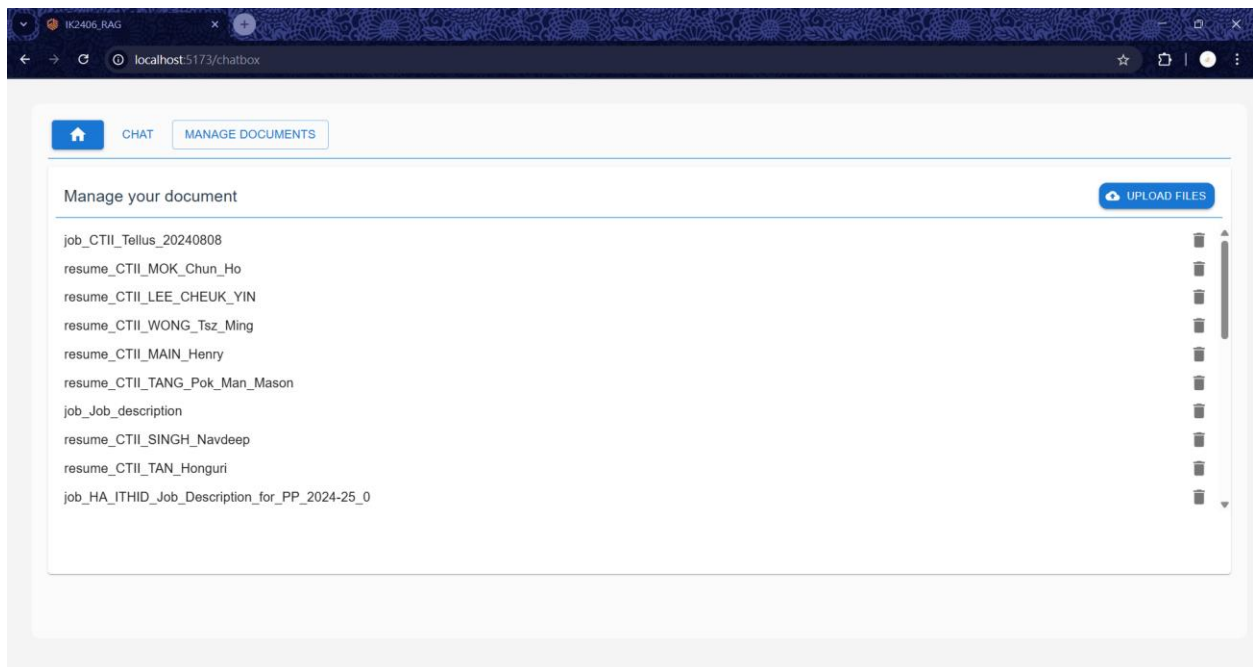


Fig 16

When navigating to the chat tab, a new conversation is automatically initiated, displaying a

unique conversation ID. Users can start chatting immediately without the need to apply for any documents.

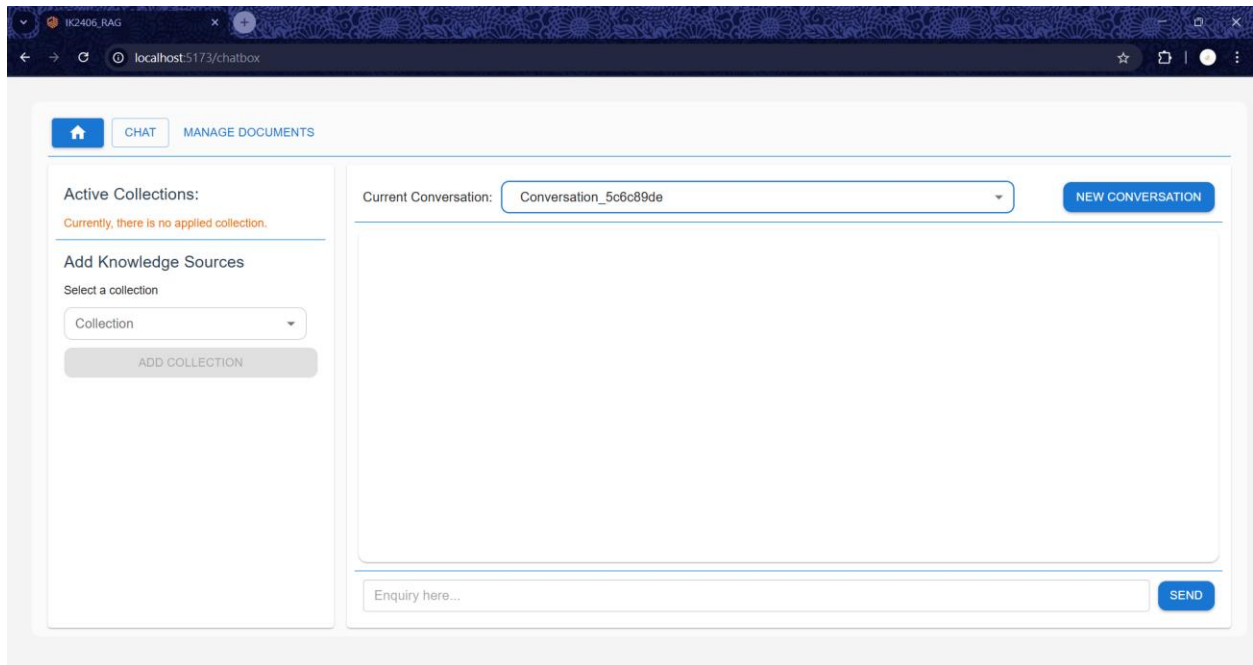


Fig 17

Additionally, users have the option to retrieve past chat histories by selecting from a list of conversation IDs. This allows them to seamlessly continue previous conversations. A new conversation can be started at any time by simply pressing the "New Conversation" button.

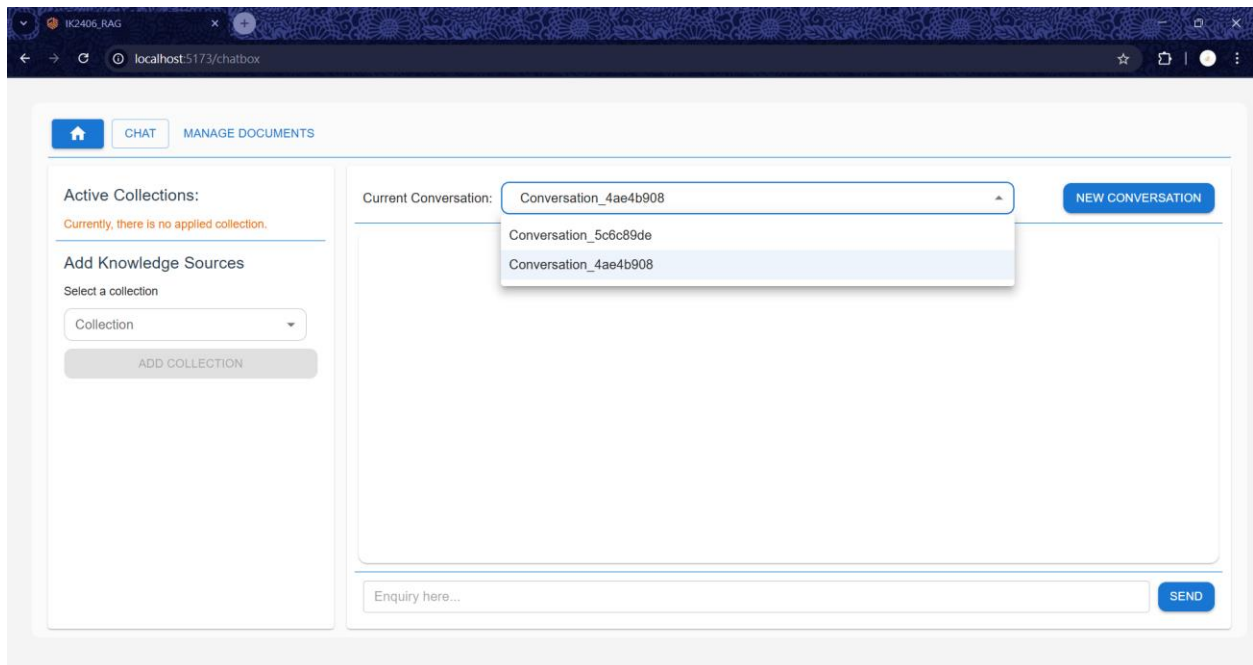


Fig 18

On the left-hand side, users can manage the documents applied to the current conversation. Uploaded documents stored in the database can be added to the current collection, with a maximum limit of 20 documents. This limit ensures optimal performance, as a larger context could increase the duration required to generate responses.

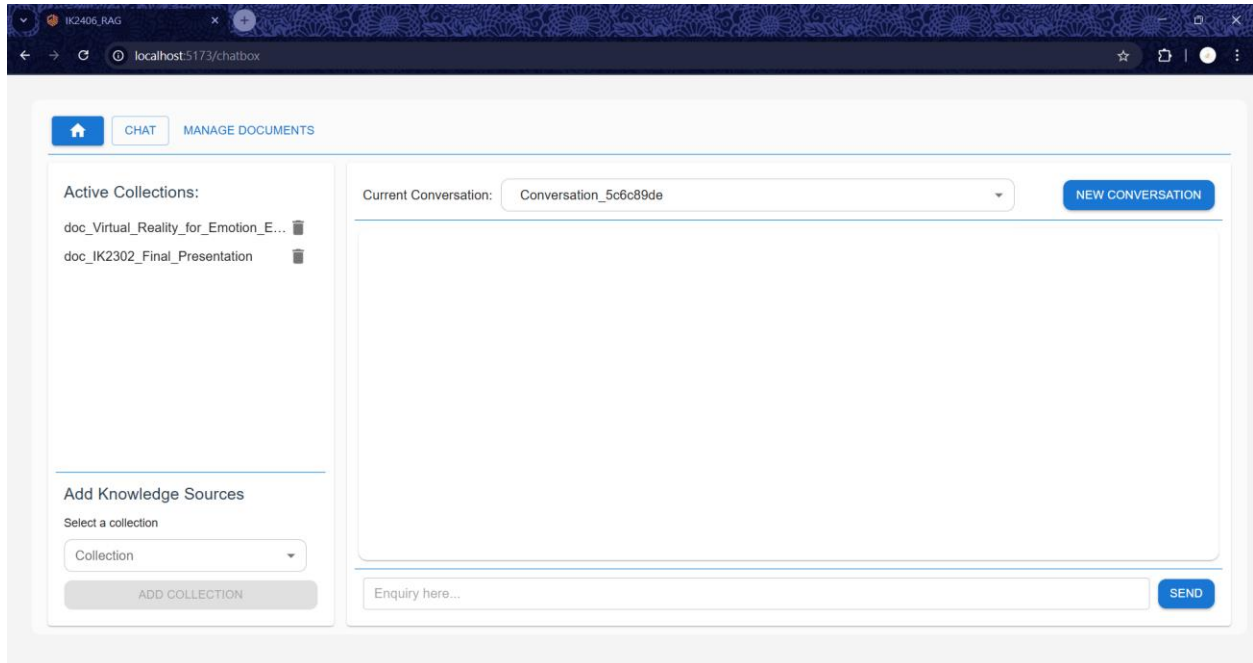


Fig19

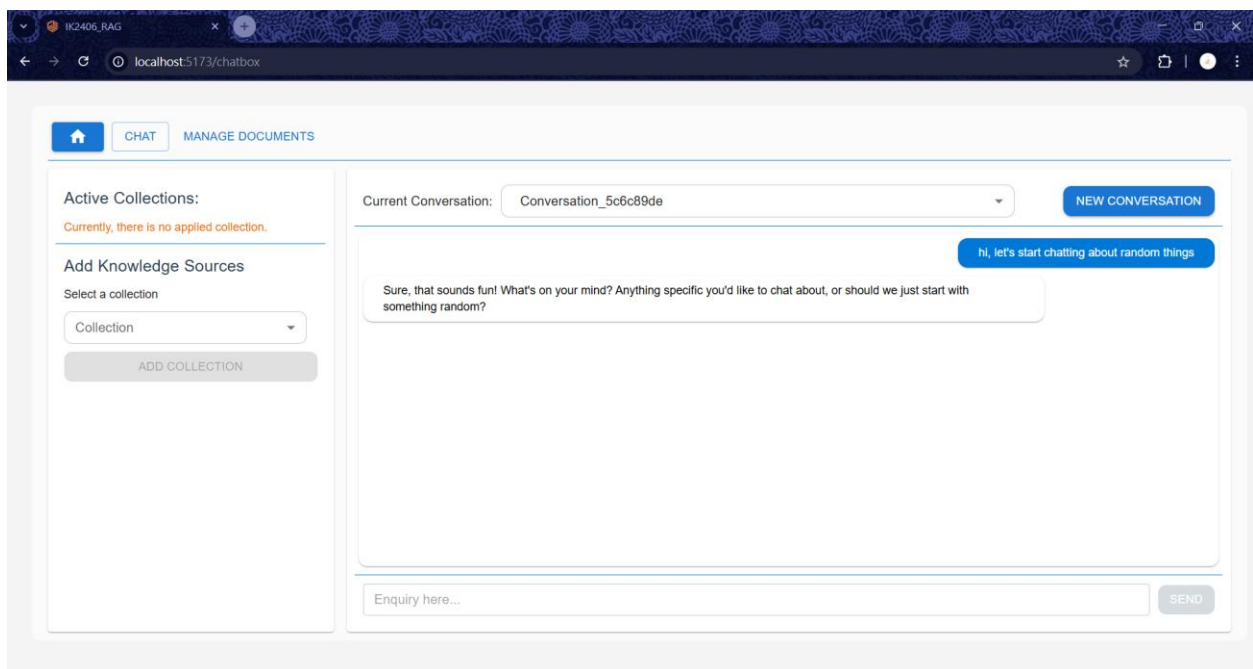


Fig 20

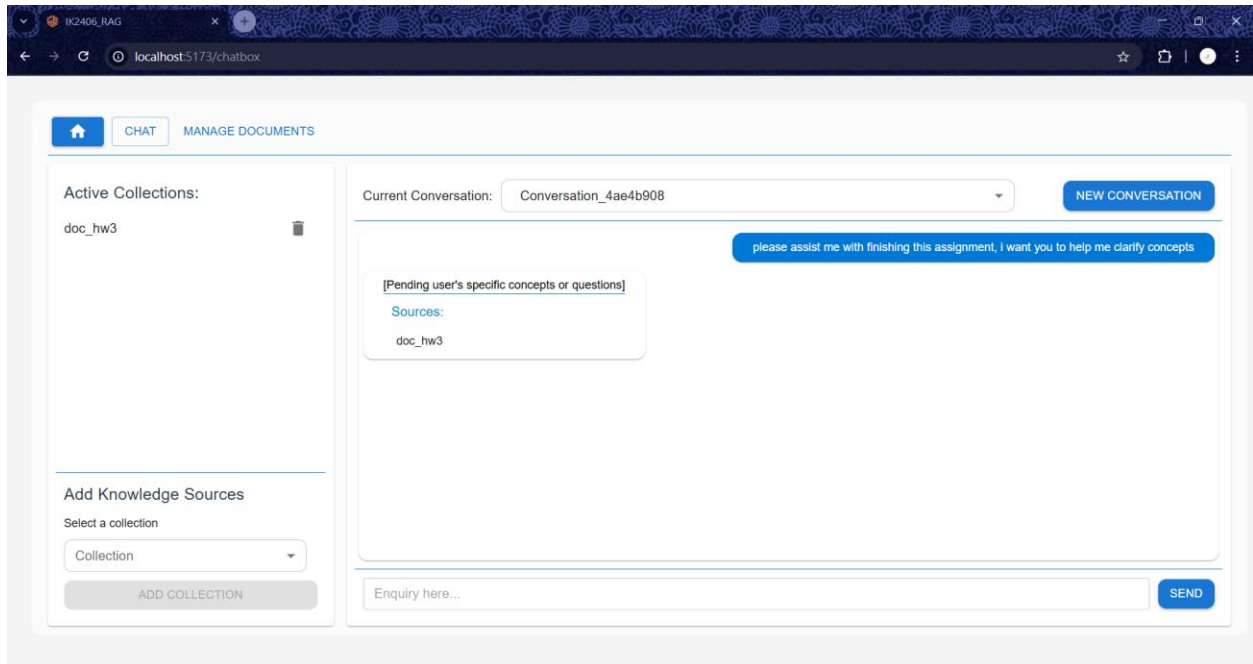


Fig 21

Furthermore, the system's replies are parsed and displayed in a well-formatted structure, enhancing readability and improving the overall user experience.

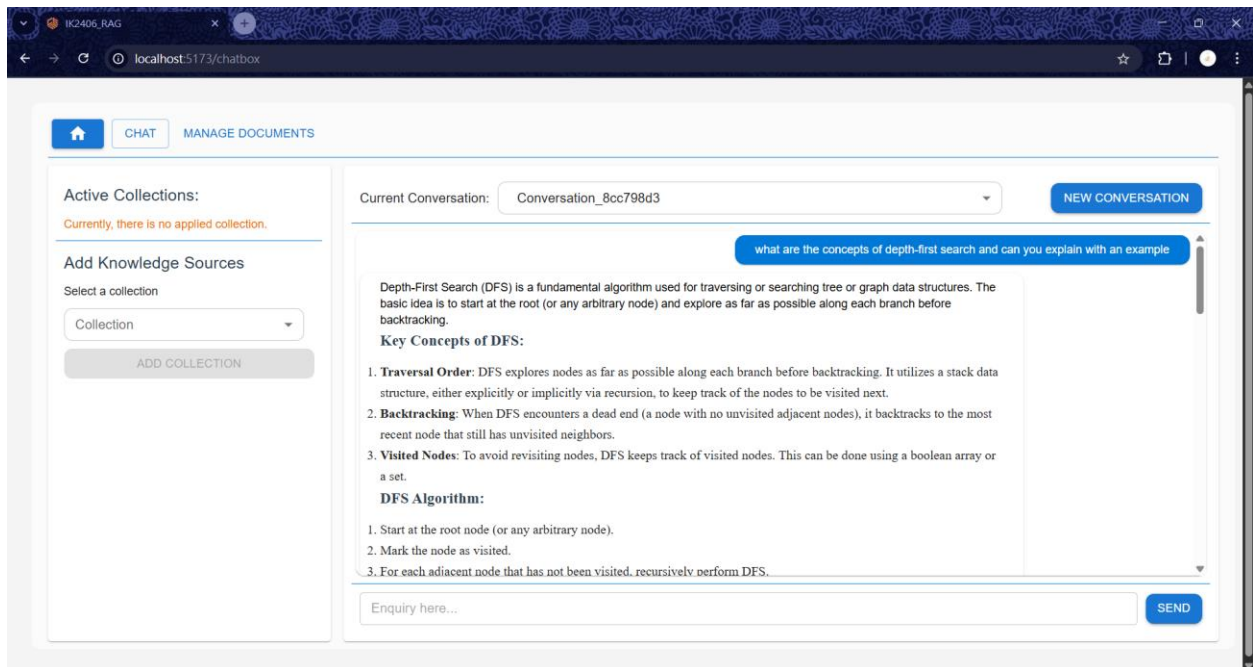


Fig 22

When users navigate to the resume screening page, the interface is divided into two columns: the left column is designated for job description analysis, and the right column is dedicated to resume screening. To activate any functionality on this page, users are required to upload a job

description. This ensures that the integrated system has the necessary input to proceed with the analysis and screening tasks.

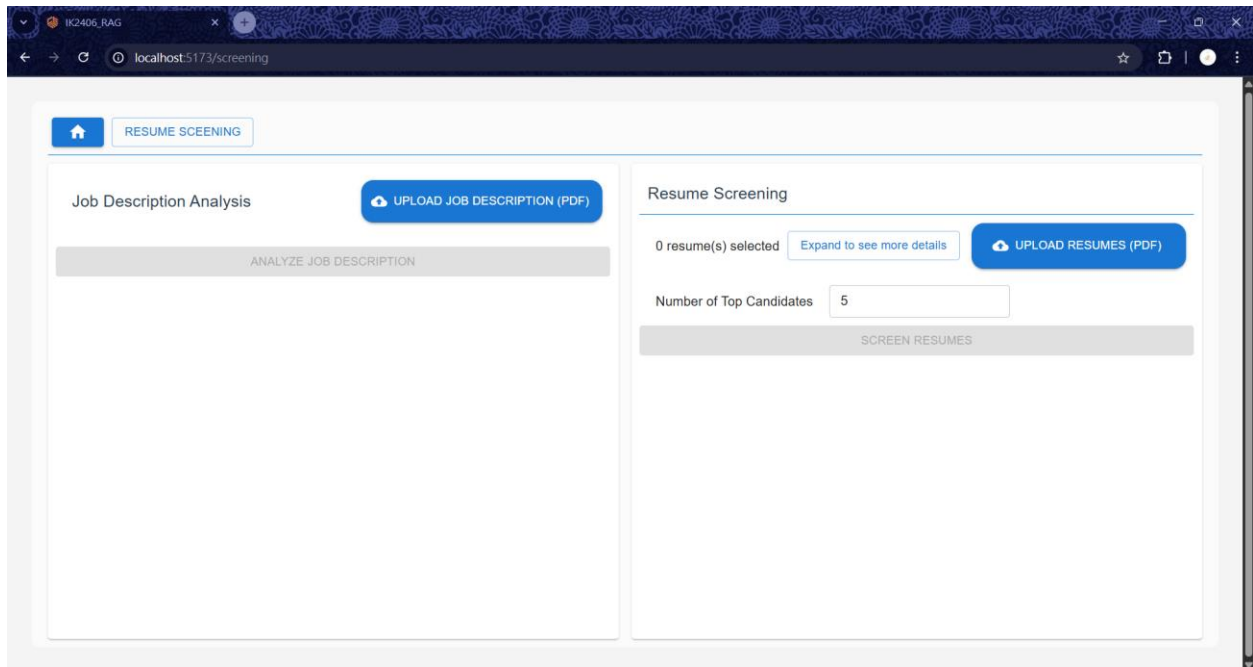


Fig 23

After uploading the job description (JD) and pressing the "Analyze Job Description" button, the system processes the JD to extract key features and requirements. Simultaneously, it suggests the weighting for each feature or requirement, providing insights into their relative importance for the hiring position. This analysis helps streamline the resume screening process by establishing clear evaluation criteria.

Users have the flexibility to modify the suggested parameter list according to their preferences. They can adjust the features or requirements and their corresponding weightings to better align with the specific needs of the hiring position. This customization ensures a more tailored and effective resume screening process.

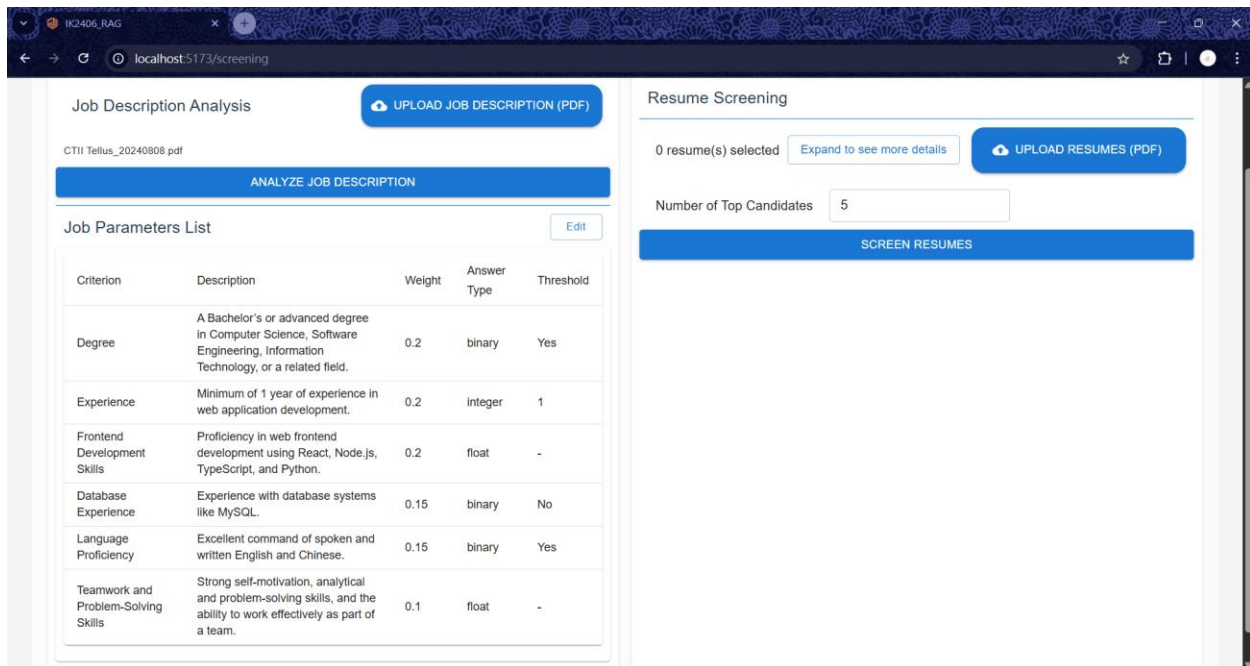


Fig 24

Users will then upload resumes to the right column of the resume screening page. This column includes a collapsible button that allows users to show or hide the list of previously uploaded resumes. Once the resumes are uploaded and the list is configured, users can press the "Screen Resumes" button. The system will then perform the screening based on the job description analyzed in the left column, ensuring an efficient and streamlined evaluation process.

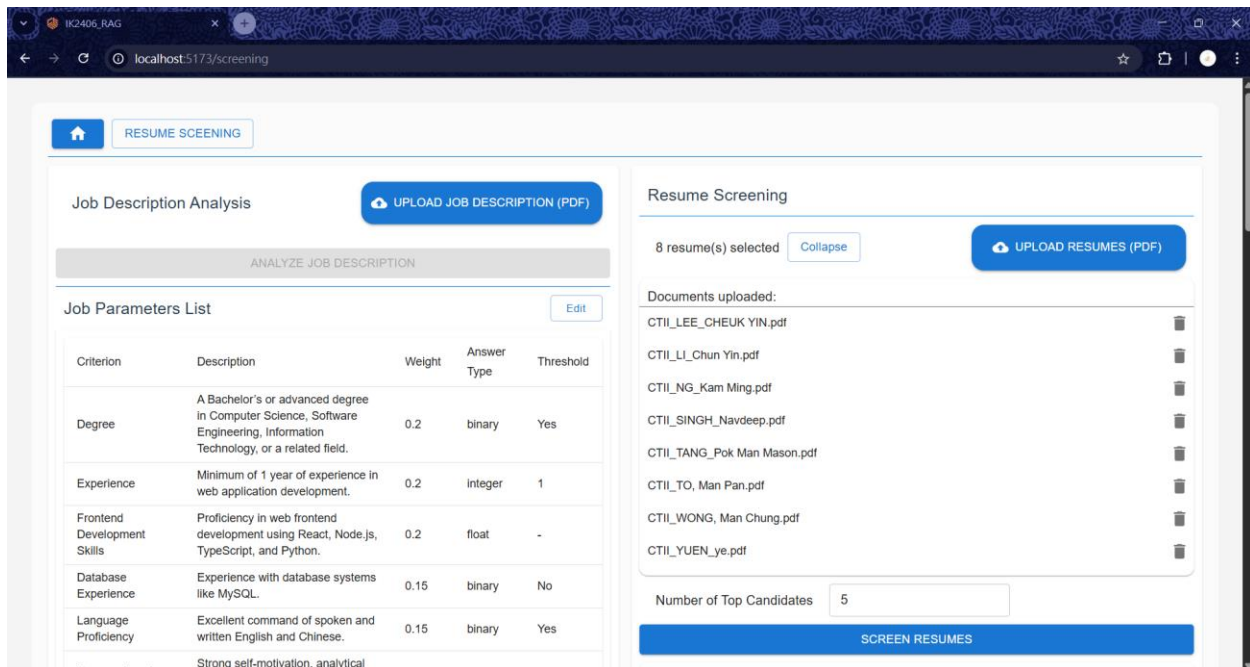


Fig 25

Once the resume screening is completed, the results are displayed in a table that highlights the top candidates. The table includes details about the candidates' qualifying or disqualifying reasons, along with their scores calculated for each criterion. Additionally, a more comprehensive explanation is provided in paragraph form beneath the table, offering deeper insights into the reasoning behind each candidate's evaluation and their suitability for the position.

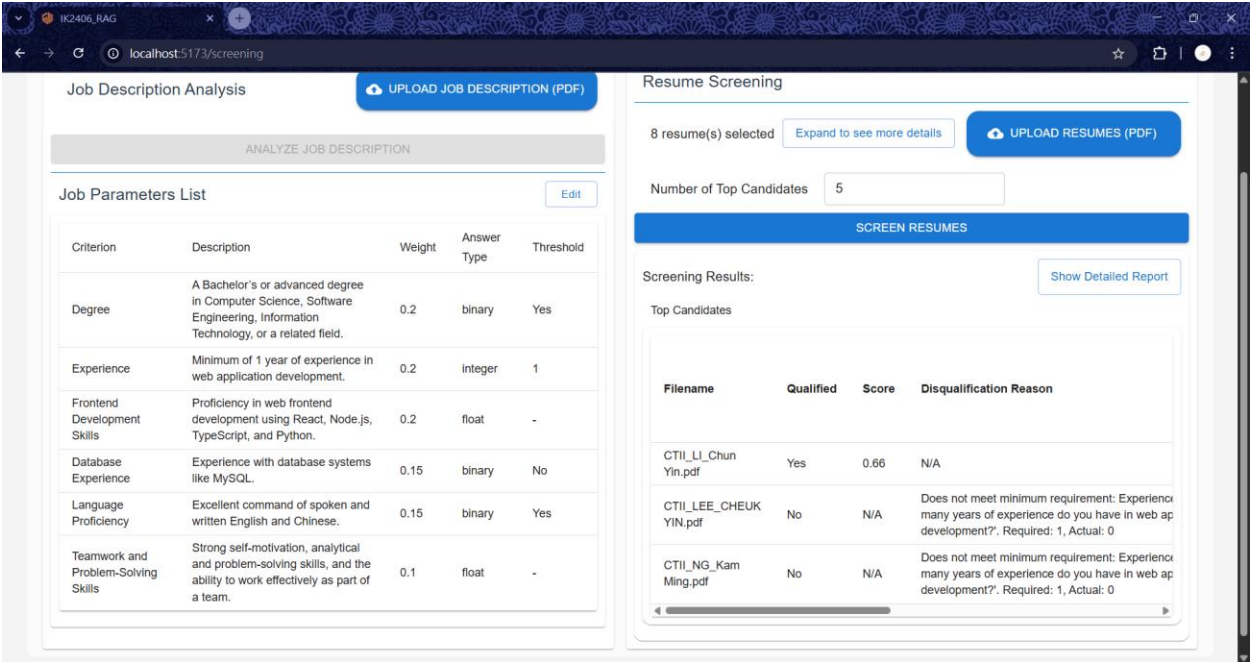


Fig26

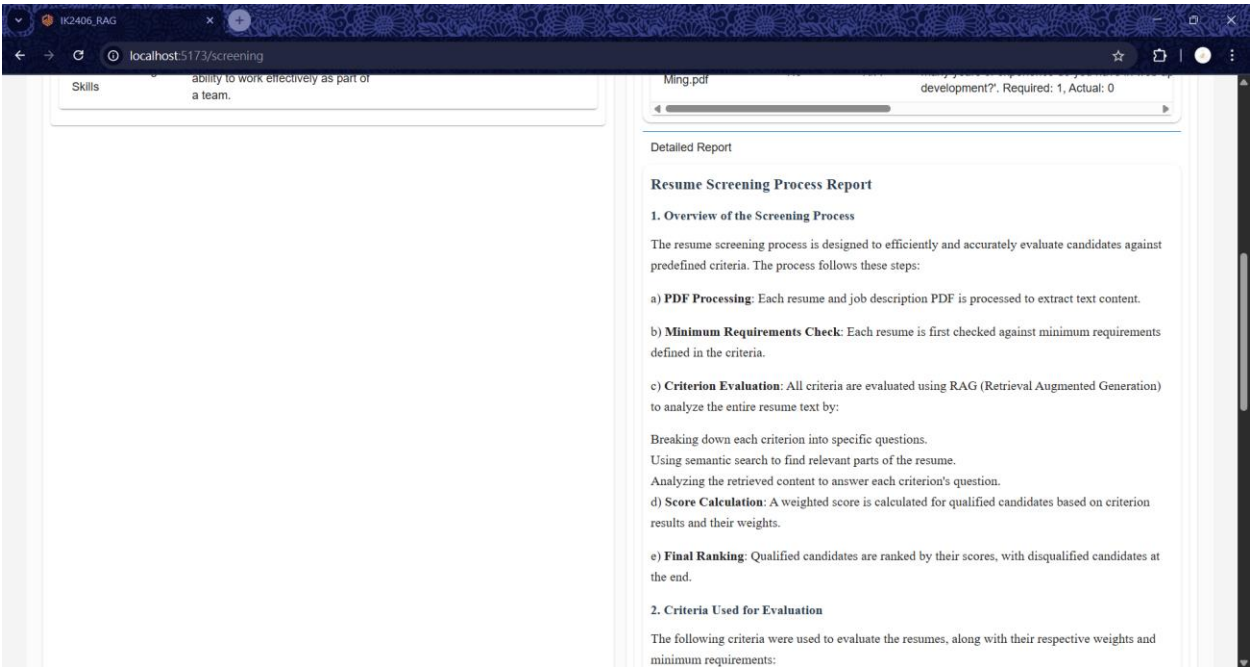


Fig 27

4.4. Backend development

The evaluated RAG system with Pre-retrieval + CoT is implemented in a FastAPI program. Backend design features other than functionalities of the RAG system:

4.4.1. Document Management

Endpoints for uploading, viewing, and deleting documents stored in the general database.

4.4.2. Collection Management for Conversations

Endpoints to manage the documents applied in active conversations, ensuring smooth integration between uploaded files and chatbot interactions.

The user can upload (or delete after uploads) PDF files, each PDF file is converted into a Chroma collection (a collection of embedded vectors) and stored within the backend server, then s/he can add or remove collections that are used as knowledge base for the conversation. The reason for such an approach instead of inserting all files into a single collection is for improving performance, different combinations of files can be easily mixed without re-creating a distinct collection through embedding for each conversation.

Chroma collection names are the same as the names of PDF files uploaded, except being sanitized to meet ChromaDB requirements, such as “starts and ends with alphanumeric”, “contains only alphanumeric, underscores or hyphens”, “no consecutive periods”, etc.

When no collections are added to the current conversation, the user can still interact with the chatbot LLM (GPT-4o) but with no RAG, otherwise RAG would be automatically used.

4.4.3. Conversation Management

Endpoints for initiating new conversations, retrieving past chat histories, and fetching answers to user queries through the integrated system.

The user can create conversations, each with an ID, for storing conversation history with the chatbot within the backend server.

The retrieval process is slightly modified compared with the initial system. Pre-retrieval with

reciprocal rank fusion is still implemented but for each collection, top k (which is 6 decided in the last semester) chunks of text would be retrieved from each collection, then after querying is performed from all collections, reciprocal rank fusion is used again to select only top k (6) chunks of text as context that would be combined with the user query.

4.4.4. Resume Screening Logic Design

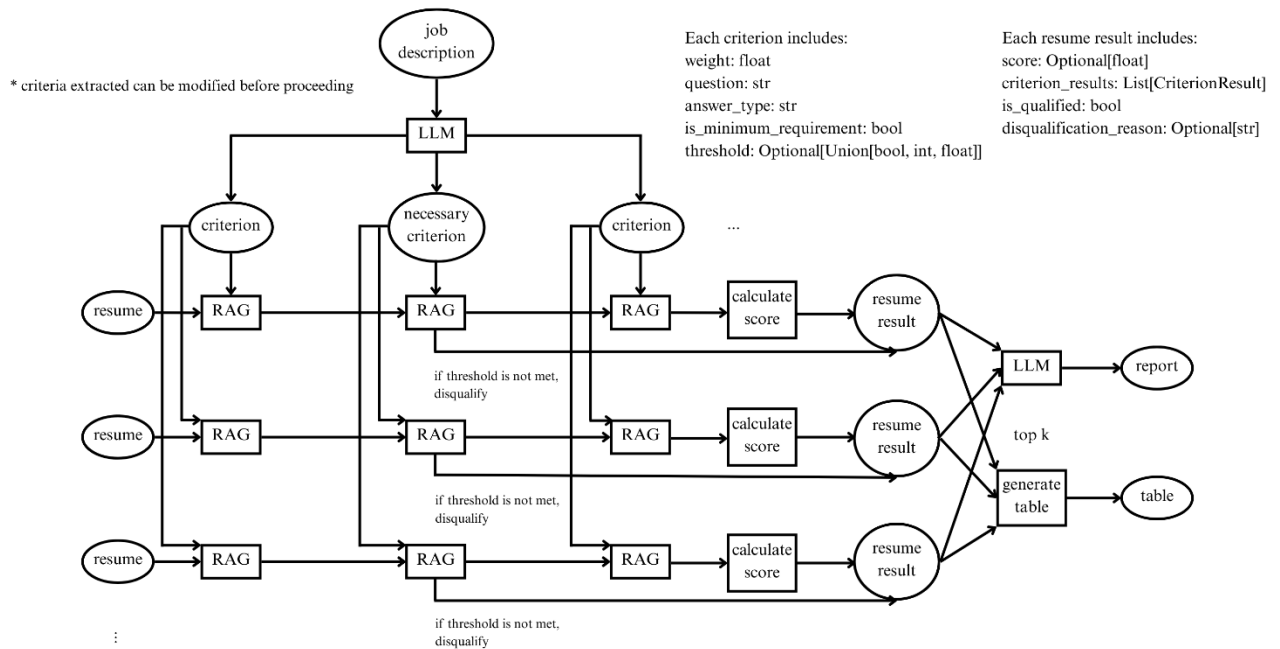


Fig 28

The resume screening process follows a structured workflow:

Feature Extraction: A PDF file of job description is sent to an "/analyze-job" endpoint. In the endpoint, text would first be extracted from the PDF file. The extracted text would then be combined with a designed prompt to extract a list of job criteria in JSON objects by a LLM (GPT-4o).

Each criterion object has several keys, including "question" which would be a question sent to RAG system with a resume as knowledge base to ask about the job candidate, "answer_type" which would be the expected type of answer to the question (binary, integer, or floating point number), "weight" which is between 0 to 1 and the sum of weights from all generated criteria would be equal to 1, "is_minimum_requirement" which indicates that it is a necessary criterion if

true, and “threshold” which is the least value of the answer that needs to be passed if it is a necessary criterion (otherwise “threshold” would be null).

```
{
  "name": "Degree",
  "description": "Bachelor's or advanced degree in Computer Science, Software Engineering, Information Technology, or a related field.",
  "weight": 0.2,
  "question": "Do you have a Bachelor's or advanced degree in Computer Science, Software Engineering, Information Technology, or a related field?",
  "answer_type": "binary",
  "is_minimum_requirement": true,
  "threshold": true
}
```

Below is an example of a criterion object: The endpoint would return a list of JSON objects.

Resume Evaluation: The user can modify values of the generated list of criteria in the user interface before proceeding. After the criteria is confirmed by the user, the JSON objects along with a “top_k” value (number of top candidates to be selected) and PDF files of resumes can be sent to an “/screen-resumes” endpoint.

Necessary Criteria: For criteria deemed mandatory (e.g., required experience or skills), candidates who lack these qualifications are automatically disqualified.

Sufficient Criteria: Criteria such as proficiency of skills or years of working experience are assigned specific weights. These are evaluated based on the candidate’s resume, and scores are calculated accordingly.

Final Scoring: In the endpoint, there is a nested for-loop, each resume would be processed sequentially, and each criterion would be assessed by RAG system sequentially for each resume. Each resume would be uploaded and converted into a Chroma collection. For each criterion assessment, the respective resume would be the only collection added as knowledge base, and questions in the criterion object would be sent to the RAG system to obtain an answer. If the criterion is a necessary criterion and the obtained answer does not meet the threshold, the candidate would be disqualified without further assessments.

For each resume without disqualification, after all criterion assessments, a score would be calculated. A binary answer would be treated as 1 as true and 0 as false. An integer answer would be normalized to a 0.0-1.0 scale. A floating-point answer would already be between 0.0 and 1.0. Each criterion answer would be multiplied with its respective weight, then being summed up together as the final score.

After calculating the score, a resume result object would be obtained for a resume, each has several keys, including “score” which is the calculated score, “criterion_results” which is a list of answers obtained from RAG system with the criterion objects, “is_qualified” which is a boolean value of whether the candidate is qualified, and “disqualification_reason” which is a string f"Does not meet minimum requirement: {criterion.name} - Question: '{criterion.question}'. Required: {criterion.threshold}, Actual: {answer}" (answer is the one obtained from RAG system).

With all resume result objects obtained, they are ranked according to their scores, and only top k (one of the inputs to the endpoint) are selected. The selected objects are separately sent to a generate_report() function and a generate_results_table() function. In generate_report(), a LLM (GPT-4o) would be called with a designed prompt to output a report which includes 1. Overview of the screening process, 2. Criteria used for evaluation, 3. Explanation of how different answer types were interpreted for scoring, 4. Summary of top candidates. And generate_results_table() is a function of concatenating the result values in a certain format.

4.4.5. Evaluation of Resume Screening System

To establish a baseline evaluation, the system utilized the "recruitment dataset" available on Kaggle. The dataset is formatted as a .csv file, with each row containing:

- Job Applicant Name – Identifies the applicant.
- Resume – Detailed text outlining qualifications and experience.
- Job Description – Role-specific criteria for evaluating candidates.
- Best Match (1/0) – Binary classification; 1 indicates the applicant is qualified, while 0 indicates disqualification

Example:

Job Applicant Name	Resume	Job Description	Best Match
Daisuke Mori	Proficient in Injury Prevention, Motivation, Nutrition, Health Coaching, Strength Training, with mid-level experience in the field. Holds a Bachelor’s degree. Holds certifications such as Certified Personal Trainer (CPT) by	A Fitness Coach is responsible for helping clients achieve their fitness goals by designing and leading group or individual fitness programs. You will provide instruction on exercises, proper form, and injury prevention techniques, encouraging clients to push their limits while maintaining a focus on their well-being. The role requires a passion for health and fitness, a strong understanding of exercise physiology, and the ability to motivate and inspire others.	0

	NASM. Skilled in delivering results and adapting to dynamic environments.	You will also monitor clients' progress and make adjustments to their fitness plans as needed to ensure continuous improvement.	
--	---	---	--

Evaluation Methodology

The Resume Screening System predicts whether an applicant is qualified and compares its results against the dataset's Best Match label. Each test case is evaluated based on the following criteria:

Qualified	Best Match	Matched with dataset
True	True	True
True	False	False
False	True	False
False	False	True

The system achieved a 68% match rate across 100 test cases.

Potential Improvements

To enhance evaluation accuracy, future efforts should:

- Expand testing with a larger dataset.
- Incorporate alternative benchmarks to improve classification validity.
- Explore resume ranking metrics for more nuanced assessments of candidate suitability.

5. Future Directions

For the future direction of this application, several enhancements and improvements can be explored.

On the system side, advanced techniques like Graph of Thought [28] and Mixture of Experts (MoE)[29] could be investigated to further optimize performance. Since there is no definitive solution for improving the system, continuous experimentation and refinement remain essential to address areas for enhancement.

For the front-end, the components can be divided into smaller, modular units to improve maintainability, especially for reusable components. Additionally, reports of screened resumes could be processed into downloadable formats such as PDF or Word documents, though the exact format remains to be finalized. Transitioning from raw HTML handling to structured form management would also enhance reliability and usability. Furthermore, as the interface has not yet been tested or evaluated by public users, collecting user feedback could guide further improvements in user experience.

On the backend, significant optimizations are possible. Multithreading could replace the current "for loop" mechanism in the resume screening process, increasing processing speed. Feedback on progress during job description or resume processing could also be incorporated, providing users with real-time updates rather than simple HTTP status codes like 200 or 500. Additionally, edge case handling needs to be strengthened, and stricter type validation should be enforced to minimize vulnerabilities and enhance system reliability.

For job criteria and resume results extracted and determined, they can be stored in an external database instead of in-memory for persistence and future access. These improvements across all aspects of the system would not only enhance functionality but also ensure a more robust and user-friendly application.

6. Conclusion

In conclusion, this application represents a significant advancement in intelligent systems by integrating innovative solutions across multiple functionalities, such as chatbot interactions and resume screening. Its robust architecture ensures enhanced relevance, contextual awareness, and efficiency, while the front-end and backend designs prioritize usability, maintainability, and performance optimization. Despite its current achievements, there remains ample scope for further improvements in terms of system techniques, user experience, processing speed, and security measures. By continually refining and benchmarking its features, this application has the potential to set new standards in intelligent system applications and deliver even greater value to its users.

7. Capstone

This project is more than just an implementation—it encapsulates my four-year journey in computer science, reflecting the evolution of my skills, mindset, and approach to problem-solving.

Year 1: The Fascination with Computing

In the beginning, I was a novice, navigating the basics of programming languages with a surface-level understanding of syntax and mathematical foundations.

My first exposure to Large Language Models (LLMs) was purely as a user, marveling at their power without grasping their underlying mechanics—like how I initially approached coding, understanding commands but not their deeper logic.

Year 2: Opening the Doors to Computer Science

With courses like ENGG2020 Digital Logic and Systems and CSCI2100 Data Structures, I started to grasp the essence of computing—how logic forms the backbone of programming and how structured data manipulation shapes efficient solutions.

This phase was comparable to reading documentation on LLM architecture, uncovering the logic behind frameworks and understanding their systematic behavior.

Year 3: Practical Applications & Engineering Mindset

This year marked a shift from theory to practice.

CSCI3100 Software Engineering introduced project management fundamentals, teaching me version control, UI/UX design, and robustness—all of which shaped my approach when building applications in this project.

CSCI3160 Design and Analysis of Algorithms emphasized elegant, optimized solutions—a mindset that resonates in crafting efficient logic.

CSCI4140 Open-source Software Development trained me to quickly adopt new frameworks, a crucial skill for integrating technologies into this project.

Year 4: Foundations Strengthened in Networking & Security

In the final year, deeper knowledge of networking and cybersecurity further cemented my expertise:

CSCI4180 Introduction to Cloud Computing and Storage introduced scalable infrastructure, crucial for deployment.

CSCI4430 Data Communication and Networks provided insights into low-level interactions, essential for debugging complex systems.

CSCI4130 Introduction to Cyber Security reinforced defensive programming—a mindset that shaped error handling and secure data access in this project.

A Concept That Stands the Test of Time: Greedy Algorithms

One concept that will always resonate with me is greedy algorithms, introduced in CSCI3160. The idea that choosing the best local decision at every step does not always lead to the best overall solution parallels the challenges we faced in this project.

Many decisions, whether in implementation strategies or logic design, were made without complete foresight, reflecting the reality of real-world development where experience and iteration are required to refine solutions. However, this does not invalidate our initial approaches. Instead, they serve as steppingstones for improvement, aligning with the essence of continuous learning and iteration.

Final Thoughts

This project represents not just a technical achievement but a reflection of personal and intellectual growth. It stands as a learning experience, proving that hard work, adaptability, and refinement over time will eventually pay off.

Github repository

System Development: <https://github.com/KEEP-EDU-HK/vg-rageval.git>

Intergrated Implementation: <https://github.com/KEEP-EDU-HK/vg-integrated-rag-system.git>

Application Development: <https://github.com/KEEP-EDU-HK/vg-rag-application.git>

References

- [1] Z. Xu, S. Jain, and M. Kankanhalli, “Hallucination is Inevitable: An Innate Limitation of Large Language Models,” arXiv.org, Jan. 22, 2024. <https://arxiv.org/abs/2401.11817>
- [2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang and H. Wang., “Retrieval-Augmented Generation for Large Language Models: A Survey,” Mar. 2024. Available: <https://arxiv.org/pdf/2312.10997>
- [3] Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiangsen Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Will Jiang, Ziyu Jiang, Lingkun Kong, Brian Moran, Jiaqi Wang, Yifan Ethan Xu, An Yan, Chenyu Yang, Eting Yuan, Hanwen Zha, Nan Tang, Lei Chen, Nicolas Scheffer, Yue Liu, Nirav Shah, Rakesh Wanga, Anuj Kumar, Wen-tau Yih, Xin Luna Dong. 2024. CRAG -- Comprehensive RAG Benchmark. arXiv preprint arXiv:2406.04744.
- [4] W. X. Zhao et al., “A Survey of Large Language Models,” arXiv (Cornell University), Mar. 2023, doi: <https://doi.org/10.48550/arxiv.2303.18223>.
- [5] H. Cao, “Recent advances in text embedding: A Comprehensive Review of Top-Performing Methods on the MTEB Benchmark,” arXiv (Cornell University), May 2024, doi: <https://doi.org/10.48550/arxiv.2406.01607>.
- [6] J. Li, T. Tang, W. X. Zhao, and J.-R. Wen, “Pretrained Language Models for Text Generation: A Survey,” arXiv.org, May 24, 2021. <https://arxiv.org/abs/2105.10311>
- [7] Y. Zhu et al., “Large Language Models for Information Retrieval: A Survey.” Available: <https://arxiv.org/pdf/2308.07107>
- [8] P. Sahoo, A. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, “A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications,” Feb. 2024. Available: <https://arxiv.org/pdf/2402.07927>
- [9] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models Chain-of-Thought Prompting,” Jan. 2023. Available: <https://arxiv.org/pdf/2201.11903>
- [10] “Chatbot Arena Leaderboard - a Hugging Face Space by lmarena-ai,” *Huggingface.co*, 2024. <https://huggingface.co/spaces/lmarena-ai/chatbot-arena-leaderboard> (accessed Oct.

31, 2024).

- [11] W. Zhao *et al.*, “A Survey of Large Language Models.” Available: <https://arxiv.org/pdf/2303.18223>
- [12] “Azure OpenAI Service models - Azure OpenAI,” *Microsoft.com*, Oct. 25, 2024. <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/models?tabs=python-secure%2Cglobal-standard%2Cstandard-chat-completions> (accessed Oct. 31, 2024).
- [13] “Build a Retrieval Augmented Generation (RAG) App |   LangChain,” *Langchain.com*, 2023. <https://python.langchain.com/docs/tutorials/rag/>
- [14] “Building Performant RAG Applications for Production - LlamaIndex,” *docs.llamaindex.ai*. https://docs.llamaindex.ai/en/stable/optimizing/production_rag/
- [15] “AICrowd | Meta KDD Cup 24 - CRAG - End-to-End Retrieval-Augmented Generation | Challenges,” AICrowd | Meta KDD Cup 24 - CRAG - End-to-End Retrieval-Augmented Generation | Challenges, 2024. <https://www.aicrowd.com/challenges/meta-comprehensive-rag-benchmark-kdd-cup-2024/problems/end-to-end-retrieval-augmented-generation> (accessed Oct. 31, 2024).
- [16] Shahul Es, Jithin James, Luis Espinosa-Anke, Steven Schockaert. RAGAS: Automated Evaluation of Retrieval Augmented Generation. 2023. arXiv preprint arXiv:2309.15217.
- [17] “Context Precision | Ragas,” *Archive.org*, 2023. https://web.archive.org/web/20240812014617/https://docs.ragas.io/en/stable/concepts/metrics/context_precision.html (accessed Oct. 31, 2024).
- [18] “Context Recall | Ragas,” *Archive.org*, 2023. https://web.archive.org/web/20240812014944/https://docs.ragas.io/en/stable/concepts/metrics/context_recall.html (accessed Oct. 31, 2024).
- [19] L. Zheng, W. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez and I. Stoica, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.” Accessed: May 05, 2024. [Online]. Available: <https://arxiv.org/pdf/2306.05685>
- [20] “prompts/templates.py · master · AICrowd / Challenges / Meta Comprehensive RAG Benchmark - KDD Cup 2024 / Meta Comprehensive RAG Benchmark starter kit · GitLab,” *GitLab*, Mar. 30, 2024. <https://gitlab.aicrowd.com/aicrowd/challenges/meta-comprehensive-rag-benchmark-kdd-cup-2024/meta-comprehensive-rag-benchmark-starter-kit/-/blob/master/prompts/templates.py> (accessed Oct. 31, 2024).
- [21] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, Nils Reimers. MTEB: Massive Text Embedding Benchmark. 2023. arXiv preprint arXiv:2210.07316v3.

- [22] “MTEB Leaderboard - a Hugging Face Space by mteb,” huggingface.co.
<https://huggingface.co/spaces/mteb/leaderboard>
- [23] Z. Rackauckas, “Rag-Fusion: A New Take on Retrieval Augmented Generation,” International Journal on Natural Language Computing, vol. 13, no. 1, pp. 37–47, Feb. 2024, doi: <https://doi.org/10.5121/ijnlc.2024.13103>.
- [24] “Build a Question Answering application over a Graph Database |  LangChain,” Langchain.com, 2024. <https://python.langchain.com/docs/tutorials/graph/> (accessed Oct. 31, 2024).
- [25] “Constructing knowledge graphs |  LangChain,” Langchain.com, 2024. https://python.langchain.com/v0.1/docs/use_cases/graph/constructing/ (accessed Oct. 31, 2024).
- [26] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, “Measuring and Narrowing the Compositionality Gap in Language Models,” arXiv.org, 2022. <https://arxiv.org/abs/2210.03350> (accessed Oct. 31, 2024).
- [27] P. Radhakrishnan et al., “Knowing When to Ask -- Bridging Large Language Models and Data,” arXiv.org, 2024. <https://arxiv.org/abs/2409.13741> (accessed Oct. 31, 2024).
- [28] M. Besta et al., “Graph of Thoughts: Solving Elaborate Problems with Large Language Models,” arXiv.org, Aug. 21, 2023. <https://arxiv.org/abs/2308.09687> (accessed Aug. 22, 2023).
- [29] W. Cai, J. Jiang, F. Wang, J. Tang, S. Kim, and J. Huang, “A Survey on Mixture of Experts,” arXiv (Cornell University), Jun. 2024, doi: <https://doi.org/10.48550/arxiv.2407.06204>.

Useful Sources

- [30] “ Usage Guide | Chroma Docs,” docs.trychroma.com. <https://docs.trychroma.com/guides>
- [31] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection,” arXiv.org, Oct. 17, 2023. <https://arxiv.org/abs/2310.11511>
- [32] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions,” *arXiv:2212.10509 [cs]*, Dec. 2022, Accessed: Apr. 03, 2023. [Online]. Available: <https://arxiv.org/abs/2212.10509>
- [33] Z. Shi, W. Sun, S. Gao, P. Ren, Z. Chen, and Z. Ren, “Generate-then-Ground in Retrieval-Augmented Generation for Multi-hop Question Answering,” arXiv.org, 2024. <https://arxiv.org/abs/2406.14891> (accessed Oct. 31, 2024).

- [34] A. Hogan et al., “Knowledge Graphs.” Accessed: May 16, 2024. [Online]. Available: <https://arxiv.org/pdf/2003.02320>
- [35] Natasha F. Noy, Yuqing Gao, Anshu Jain, Anant Narayanan, Alan Patterson, and Jamie Taylor. 2019. Industry-scale Knowledge Graphs: Lessons and Challenges. *ACM Queue* 17, 2 (2019), 20
- [36] Serge Abiteboul. 1997. Querying Semi-Structured Data. In *Database Theory - ICDT '97*, 6th International Conference, Delphi, Greece, January 8-10, 1997, Proceedings (Lecture Notes in Computer Science), Foto N. Afrati and Phokion G. Kolaitis (Eds.), Vol. 1186. Springer, 1–18. https://doi.org/10.1007/3-540-62222-5_33
- [37] M. Raj, J. Heltech Bengaluru, H. Warriar, Y. Gupta, and H. Bengaluru, “FINE TUNING LLMS FOR ENTERPRISE: PRACTICAL GUIDELINES AND RECOMMENDATIONS.” Available: <https://arxiv.org/pdf/2404.10779>
- [38] E. Hu et al., “LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS,” Oct. 2021. Available: <https://arxiv.org/pdf/2106.09685>
- [39] L. Xu, H. Xie, S.-Z. Qin, X. Tao, and F. Wang, “Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment.” Available: <https://arxiv.org/pdf/2312.12148>
- [40] S. Setty, K. Jijo, E. Chung, and N. Vidra, “Improving Retrieval for RAG based Question Answering Models on Financial Documents,” arXiv.org, Mar. 22, 2024. <https://arxiv.org/abs/2404.07221>
- [41] Q. Jin, Y. Yang, Q. Chen, and Z. Lu, “GeneGPT: Augmenting large language models with domain tools for improved access to biomedical information,” *Bioinformatics*, Feb. 2024, doi: <https://doi.org/10.1093/bioinformatics/btae075>.
- [42] D. Hennekeuser, Daryoush Daniel Vaziri, D. Golchinfar, D. Schreiber, and G. Stevens, “Enlarged Education – Exploring the Use of Generative AI to Support Lecturing in Higher Education,” *International Journal of Artificial Intelligence in Education*, Aug. 2024, doi: <https://doi.org/10.1007/s40593-024-00424-y>.
- [43] “Starter Tutorial (OpenAI) - LlamaIndex,” *Llamaindex.ai*, 2021. https://docs.llamaindex.ai/en/stable/getting_started/starter_example/ (accessed Oct. 31, 2024).